Exercise 6

Joshua John E. Santiago

2026-11-06

#R Lab: Data Structures - Data Frames #Part A — Creating Data Frames

```
name <- c("Ana", "Ben", "Cid", "Dex", "Eli")
age <- c(20, 21, 19, 22, 20)
passed <- c(TRUE, TRUE, FALSE, TRUE, FALSE)
```

```
students <- data.frame(name = name, age = age, passed = passed)
print(students)
```

```
##   name age passed
## 1  Ana  20   TRUE
## 2  Ben  21   TRUE
## 3  Cid  19  FALSE
## 4  Dex  22   TRUE
## 5  Eli  20  FALSE
```

```
typeof(students$name)
```

```
## [1] "character"
```

```
class(students$name)
```

```
## [1] "character"
```

```
courses <- data.frame(
  course_code = c("IS101", "IS102", "IS103"),
  units = c(3, 3, 2),
  instructor = c("Dr. P. Reyes", "Dr. R. Santos", "Dr. S. Cruz")
)
print(courses)
```

```
##   course_code units    instructor
## 1       IS101     3  Dr. P. Reyes
## 2       IS102     3 Dr. R. Santos
## 3       IS103     2   Dr. S. Cruz
```

#Question #1

```
# Answer: A data frame is a table, so every column has to have the same number
# of rows. If one column is shorter, some cells would be empty and the table
# would not be a proper rectangle. R gives an error like: arguments imply
# differing number of rows: 5, 4.
```

#Question #2

```
# Answer: A data frame is built like a list, where each element is a column.
# A list can hold different types in different elements, so each column can
```

```
# have its own type. But each column is a vector, and a vector can only hold
# one type, so a single column cannot mix types.
```

#Part B — Subsetting Data Frames

```
students$age
```

```
## [1] 20 21 19 22 20
```

```
students[["age"]]
```

```
## [1] 20 21 19 22 20
```

```
students["age"]
```

```
##   age
## 1  20
## 2  21
## 3  19
## 4  22
## 5  20
```

```
students[2, ]
```

```
##   name age passed
## 2  Ben  21   TRUE
```

```
students[3, "name"]
```

```
## [1] "Cid"
```

```
students[students$passed == TRUE, ]
```

```
##   name age passed
## 1  Ana  20   TRUE
## 2  Ben  21   TRUE
## 4  Dex  22   TRUE
```

```
students[students$passed == TRUE & students$age > 19, ]
```

```
##   name age passed
## 1  Ana  20   TRUE
## 2  Ben  21   TRUE
## 4  Dex  22   TRUE
```

#Question #3

```
class(students$age)
```

```
## [1] "numeric"
```

```
class(students["age"])
```

```
## [1] "data.frame"
```

```
# Answer: students$age gives numeric, which is just the plain vector of ages.
# students["age"] gives data.frame, so it is still a small table with one
# column. This is like lists, where single brackets keep the container and
# double brackets (or $) pull out the actual content inside.
```

#Question #4

```
# Answer: The & operator compares the two logical vectors position by position.
# A row is TRUE only if both conditions are TRUE for that row, and then R keeps
# only the TRUE rows. If I used | (OR) instead, a row would be kept when at
# least one condition is TRUE, so I would get more rows. In this data, Eli
# would be added because Eli is older than 19 even though Eli did not pass.
```

#Part C — Data Frame Properties

```
str(students)
```

```
## 'data.frame':    5 obs. of  3 variables:
##  $ name  : chr  "Ana" "Ben" "Cid" "Dex" ...
##  $ age   : num  20 21 19 22 20
##  $ passed: logi  TRUE TRUE FALSE TRUE FALSE
```

```
dim(students)
```

```
## [1] 5 3
```

```
nrow(students)
```

```
## [1] 5
```

```
ncol(students)
```

```
## [1] 3
```

```
names(students)
```

```
## [1] "name"   "age"    "passed"
```

```
summary(students)
```

```
##         name                age          passed
##  Length   :5   Min.   :19.0   Mode :logical
##  N.unique :5   1st Qu.:20.0   FALSE:2
##  N.blank  :0   Median :20.0   TRUE :3
##  Min.nchar:3   Mean   :20.4
##  Max.nchar:3   3rd Qu.:21.0
##                Max.   :22.0
```

#Question #5

```
# Answer: For age (numeric), summary gives the min, 1st quartile, median, mean,
# 3rd quartile, and max. For passed (logical), it just counts how many are
# FALSE and how many are TRUE. For name (character), it only shows the length,
# class, and mode. A mean does not make sense for name because names are text,
# and you cannot add up or average text.
```

#Question #6

```
# Answer: str() is more useful when I want to see the structure, like the data
# type of each column and a preview of the values. This helps when checking if
# the data was read in correctly. summary() is more useful when I want the
# actual statistics, like the mean and median, or how many are TRUE or FALSE.
```

#Part D — Data Frame Operations

```
# Add year_level column
students$year_level <- c(2, 3, 1, 4, 2)
print(students)
```

```
##   name age passed year_level
## 1  Ana  20   TRUE          2
## 2  Ben  21   TRUE          3
## 3  Cid  19  FALSE          1
## 4  Dex  22   TRUE          4
## 5  Eli  20  FALSE          2
```

```
# Remove the passed column
students$passed <- NULL
print(students)
```

```
##   name age year_level
## 1  Ana  20          2
## 2  Ben  21          3
## 3  Cid  19          1
## 4  Dex  22          4
## 5  Eli  20          2
```

```
# Sort by age, ascending
students[order(students$age), ]
```

```
##   name age year_level
## 3  Cid  19          1
## 1  Ana  20          2
## 5  Eli  20          2
## 2  Ben  21          3
## 4  Dex  22          4
```

```
# Sort by age, descending
students[order(students$age, decreasing = TRUE), ]
```

```
##   name age year_level
## 4  Dex  22          4
## 2  Ben  21          3
## 1  Ana  20          2
## 5  Eli  20          2
## 3  Cid  19          1
```

#Question #7

```
students[students$passed == TRUE, ]
```

```
## [1] name       age        year_level
## <0 rows> (or 0-length row.names)
```

```
# Answer: No, the filter does not work anymore. Since the passed column is
# gone, students$passed gives NULL, and NULL == TRUE gives an empty logical
# vector. R does not show an error, it just returns a data frame with 0 rows.
# This is why I need to keep track of what the data frame looks like after
# each step, because the code can run without errors but give a useless
# result.
```

#Question #8

```
order(students$age)
```

```
## [1] 3 1 5 2 4
```

```
# Answer: order(students$age) gives the row positions arranged from the
# smallest age to the largest, which is 3 1 5 2 4 here. Then
```

```r
# students[that vector, ] reads it as "take row 3 first, then row 1, then row
# 5, then 2, then 4". Since the column part is left empty, R takes all the
# columns for each of those rows, so the whole rows move together and not
# just age.
```

#Part E — Critical Thinking Challenge

```r
grades <- data.frame(
  name = c("Ana", "Ben", "Cid", "Dex", "Eli"),
  midterm = c(85, 78, 92, 88, 70),
  final = c(90, 82, 95, 84, 75)
)

# Add the average column
grades$average <- (grades$midterm + grades$final) / 2

# The lab uses a remarks column, so I made one. I assumed below 75 is Failed.
grades$remarks <- ifelse(grades$average < 75, "Failed", "Passed")
print(grades)
```

```
##   name midterm final average remarks
## 1  Ana      85    90    87.5  Passed
## 2  Ben      78    82    80.0  Passed
## 3  Cid      92    95    93.5  Passed
## 4  Dex      88    84    86.0  Passed
## 5  Eli      70    75    72.5  Failed
```

```r
# Only the Failed rows, showing name and average
grades[grades$remarks == "Failed", c("name", "average")]
```

```
##   name average
## 5  Eli    72.5
```

#Question #9

```r
# Answer: First I would add a new column for the change from midterm to final,
# like grades$change <- grades$final - grades$midterm. Then I would sort the
# data by change using order() to see who dropped the most. After that I would
# filter the Failed rows and look at both columns. If most of the failed
# students have a big negative change, the drop is the main factor. If they
# have a low midterm but a normal change, then the low midterm is the main
# factor.
```

#Conclusion

```r
# Answer: A data frame is a table where each column is a vector, and all the
# columns have the same length. It is like a list of equal length vectors,
# which is why each column can have a different type but each column itself
# has only one type. It is the main structure for real data because most
# real datasets, like student records or sales, are tables with different
# kinds of information in each column. It also makes it easy to filter, sort,
# and add columns.
```